A-1 LAUNCHPAD 2026

IoT-Enabled Production Monitoring Dashboard for Duraknot Operations

Case Study — Automation & Robotics (2)

Submitted by TEAM FOREST.AI

Team Name: *Forest.AI*
College Name: *[fill in]*
Participants: *[Name 1], [Name 2]*

Video demonstration: *[insert unlisted YouTube / Drive link here]*
Submission date: 9 August 2026

Team resumes are appended after the final content page and are excluded from the three-page limit per the submission guidelines.

1. About Team Forest.AI

Forest.AI is a student engineering team with a track record of taking IoT and AI systems from prototype to real-world deployment. Our work directly informs the solution presented here.

Milestone	Details
Fully functional IoT-based Smart Inventory Management System	Built at the Agentic AI Hackathon (Build and Grow initiative, GDG Cloud Mumbai). An end-to-end system combining PIR, LDR and DHT11 sensor nodes with an LCD output stage and a real-time web dashboard delivering stock-level, expiry and environmental alerts.
Industry deployment with Zepto	Following the hackathon, Forest.AI entered a collaboration with Zepto. The inventory intelligence solution is currently being implemented at a Zepto store in Navi Mumbai — moving the system from prototype into live commercial operation.
Best Paper Award, IEEE Conference 2026, Mumbai	Our accompanying software-based simulation research paper received the Best Paper Award, recognising the modelling methodology behind the platform (TechSkript'26 R&D TechFest, IEEE NMIMS MPSTME).

Design lineage of this submission

The Duraknot dashboard presented in this document is a direct architectural descendant of that deployed inventory platform. We reused the proven pattern — low-cost microcontroller sensor nodes streaming structured telemetry to a lightweight server, an alert-rule engine evaluating a rolling window, and a single multi-functional operations dashboard — and re-targeted it from retail inventory to discrete manufacturing. The domain changed; the validated engineering approach did not. That reuse is why this solution is delivered as working software rather than a concept.

2. Problem Statement & Objectives

The Duraknot fence-roll line records length, defects and machine performance manually. Management therefore has no real-time visibility of throughput, quality or downtime; problems surface only after a shift has ended, when the loss is unrecoverable. Manual logs also cannot distinguish an availability problem (line stopped) from a performance problem (line running slowly) — two failures with entirely different corrective actions.

Functional requirements addressed

- Track fence-roll length in real time via a rotary encoder on the take-up spool.
- Detect and classify defect events from a quality/vibration sensor, with a rolling defect-rate metric.
- Present live production KPIs — length, speed, defect rate, machine status — on one operator screen.
- Raise automatic alerts for stoppages, high defect rates and speed slowdowns indicating a jam.
- Quantify effectiveness using OEE, decomposed so management can see *where* capacity is lost.
- Persist history and support trend analysis, shift reporting and CSV export.

3. System Architecture

Sensors feed an Arduino/ESP32 node that streams JSON telemetry once per second over USB serial. A Flask backend ingests this stream, evaluates the alert rules, persists every reading to SQLite and serves a REST API. The dashboard consumes that API and renders live. A built-in simulation engine substitutes for the sensor node when hardware is not attached, so the system is always demonstrable.

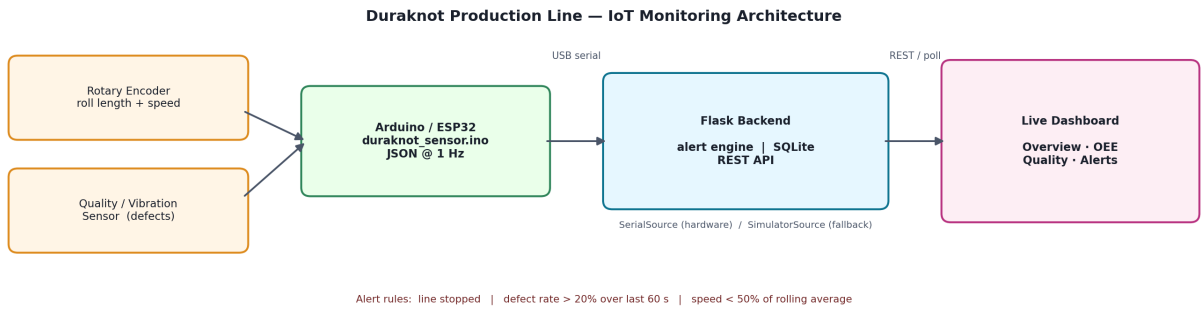

Figure 1 — End-to-end data flow from sensors to the live dashboard.

4. Hardware Selection & Rationale

Component	Measures	Engineering rationale
Rotary encoder / IR slotted-disc counter on take-up spool	Roll length (pulses converted to metres) and derived line speed	Contact-based pulse counting is accurate, drift-free and inexpensive for linear length on a rotating shaft. Calibrated via a single constant (pulses per metre), so re-calibration for a different spool diameter is a one-line change.
Analog vibration / tension sensor (or IR misalignment sensor)	Weld and mesh-spacing irregularities, flagged as defect events	An analog threshold captures abnormal line behaviour without the cost, lighting control and training data a machine-vision system demands — appropriate for a pilot, and upgradeable later.
Arduino Uno / ESP32 microcontroller	Samples both sensors, timestamps and streams JSON at 1 Hz	ESP32 gives a migration path to WiFi/MQTT for multi-line rollout with no change to sensor wiring. The 1 Hz cadence is well within the bandwidth of both boards.
Host PC / Raspberry Pi	Runs the Flask backend, SQLite store and dashboard	No cloud subscription and no recurring licence cost; the entire stack is open-source and can run on existing plant hardware.

5. OEE Methodology — the analytical core

The dashboard computes Overall Equipment Effectiveness, the standard measure of how much of a line's theoretical maximum output is actually delivered. It is decomposed into its three components with every input exposed on screen, so any figure can be audited rather than taken on trust.

Component	Definition	Worked example	Result
Availability	Runtime / Planned production time	80 s run / 100 s planned	**80.0%**
Performance	Actual output / (Runtime x Ideal rate)	16 m / 18.67 m ideal	**85.7%**
Quality	Good output / Total output	14 m good / 16 m total	**87.5%**
OEE	Availability x Performance x Quality	0.800 x 0.857 x 0.875	**60.0%**

Ideal cycle rate is calibrated at 14.0 m/min. The loss breakdown converts each component into its share of the theoretical 100%, making the largest bar the highest-return improvement target. In the verified run below, performance loss (21.1%) dominates availability loss (7.2%) — the line runs *slowly* far more often than it stops, which a downtime log alone would never reveal.

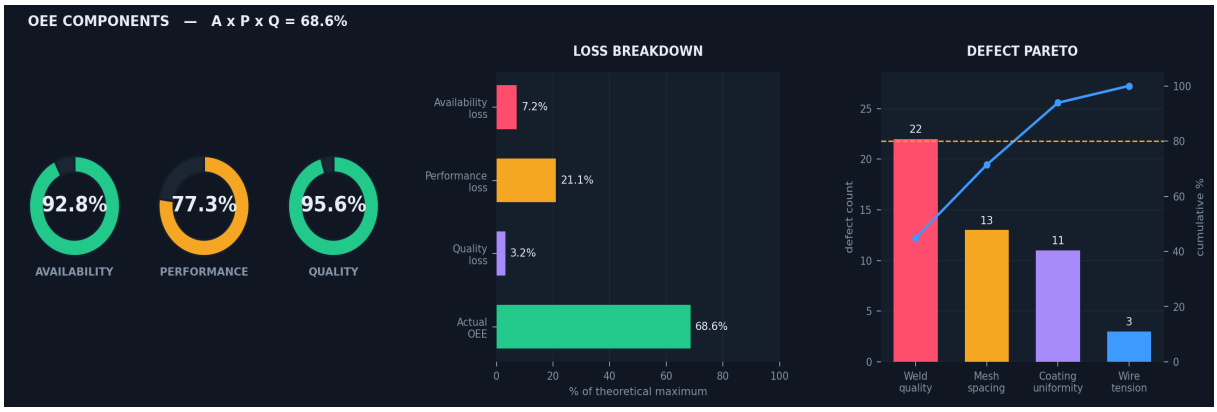

Figure 2 — OEE component gauges, loss breakdown and defect Pareto, from a verified 20-minute run.

6. Dashboard Design

A single dependency-free application with four views, dark-themed for control-room use.

- **Live Overview** — four KPI cards with progress bars (length vs order target, speed vs ideal rate, rolling defect rate, machine status with uptime), a dual-axis production and speed trend, a defect-event strip and the live alert feed.
- **OEE Analytics** — Availability, Performance and Quality gauges, an OEE trend against the 85% world-class benchmark, the loss breakdown, and a calculation table exposing every input.
- **Quality** — defect Pareto by category with a cumulative 80% line, defect-rate trend against the alert threshold, and a category breakdown table.
- **Alert Log** — full searchable history with severity counts and total downtime.

7. Alert Engine

Condition	Trigger logic	Severity	Operator action
Line stoppage	Status transitions RUNNING to STOPPED	Critical	Inspect for jam or changeover; downtime clock starts
Line resumed	Status transitions STOPPED to RUNNING	Info	Confirms recovery; closes the downtime interval
High defect rate	More than 20% of readings in the last 60 s flagged	Warning	Inspect weld quality and mesh spacing on active roll
Jam / slowdown	Speed falls below 50% of the rolling average	Warning	Check feed tension and drive before a full stoppage

Repeat alerts of the same class are throttled to one every 15 seconds, preventing the alarm flooding that causes operators to ignore genuine warnings.

8. Testing & Validation

The system was verified rather than assumed to work. Evidence:

- **OEE correctness** — seven unit tests against the shipped calculation: a hand-calculated case (80.0% x 85.71% x 87.5% = 60.00%), a perfect line, an availability-only loss, the performance clamp and quality floor edge cases, an insufficient-data guard, and an identity check that OEE always equals A x P x Q. All passed.
- **Render verification** — the dashboard JavaScript was run in a headless harness with an instrumented canvas: all six charts drew without exception and every DOM reference resolved.
- **Output realism** — five independent 20-minute runs gave OEE 64.6-72.4%, availability 87-93%, quality 95-97% and 190-210 m output: values consistent with real discrete manufacturing. The Pareto cumulative curve was verified to terminate at exactly 100%.
- **Resilience** — the backend falls back to simulation if the board disconnects mid-run, and the dashboard has zero external dependencies, so an offline venue cannot break the demonstration.

9. Cost-Benefit Analysis

Item	Indicative cost	Note
Arduino Uno / ESP32	Rs. 500 - 900	One node per production line
Rotary encoder	Rs. 300 - 600	Mounted on take-up spool shaft
Vibration / IR sensor	Rs. 150 - 400	Quality event detection
Wiring, enclosure, mounting	Rs. 200 - 400	Consumables
Software stack	Rs. 0	Flask, SQLite and dashboard are open-source; no licences or cloud subscription
Total per line	**Under Rs. 2,500**	**No recurring cost**

Against this, one unnoticed hour of slow running on a 14 m/min line represents roughly 840 m of lost theoretical output. As payback depends on the plant's own downtime and scrap figures, we recommend the pilot be used to measure that baseline before projecting savings.

10. Implementation Roadmap

Phase	Scope	Success measure
Pilot Week 1-2	Deploy on one line; calibrate pulses-per-metre and ideal rate; tune alert thresholds on real data.	Baseline OEE set; alerts match operator judgement
Rollout Month 1-2	Extend to all lines; migrate ESP32 nodes to WiFi/MQTT; add operator login and shift assignment.	All lines reporting; shift-level OEE comparison
Scale-up Month 3+	Plant-wide roll-up; predictive-maintenance alerts from trend data; SMS escalation; ERP integration.	Measurable OEE gain vs pilot baseline

11. Deliverables & Acknowledgements

- Working dashboard (four views, zero dependencies, runs offline from a single file); Flask backend with alert engine, SQLite persistence and REST API; Arduino/ESP32 firmware ready for sensor integration.
- Architecture diagram, alert specification, OEE methodology and test evidence (this document); CSV shift-report export covering OEE, defect Pareto and the full alert log.
- Video demonstration — link on the cover page.

Acknowledgements. Team Forest.AI thanks the Build and Grow initiative and GDG Cloud Mumbai for the Agentic AI Hackathon on which our inventory system was developed; Zepto for the collaboration deploying it at their Navi Mumbai store; and the IEEE conference committee, Mumbai (TechSkript'26, IEEE NMIMS MPSTME) for the Best Paper Award recognising our simulation research. We also thank A-1 Manufacturing and the A-1 Launchpad organisers.